

Confidence-Based Out-of-Distribution Detection

for Cat & Dog Image Classification

Extending Binary Classifiers to Detect Unknown Inputs via Gaussian-Aware Confidence Thresholding

TEAM MEMBERS

Prisha Gautam
Suhani Joshi

PROJECT MENTOR / TEACHING ASSISTANT

Dilip Parmar

PyTorch

EfficientNet-B0

Transfer Learning

OOD Detection

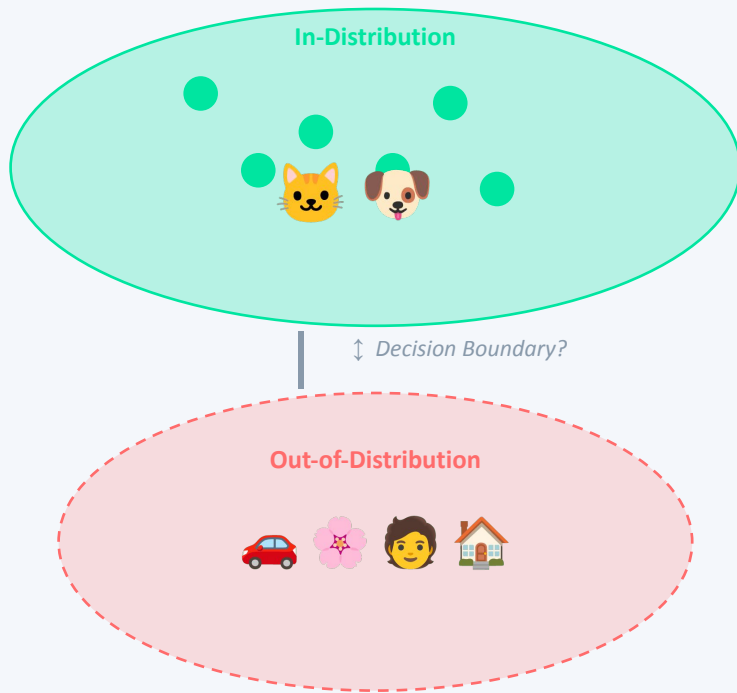
Introduction to Out-of-Distribution (OOD) Detection

What is OOD?

An OOD sample is any input that falls outside the distribution the model was trained on. Standard classifiers have no mechanism to say "I don't know" — they always force a prediction.

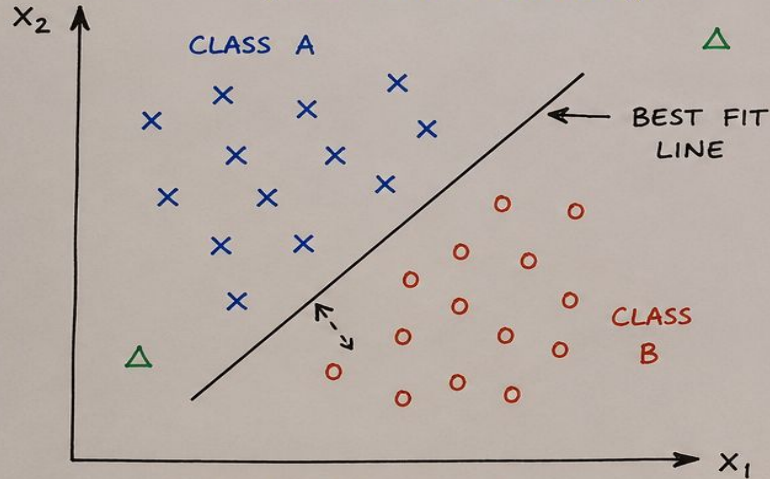
Why Does OOD Matter?

- Medical imaging: unseen disease flagged as healthy
- Autonomous driving: unknown object classified as road
- Security: adversarial inputs misclassified with high confidence



WHY OOD DETECTION IS NOT EASY

LINEAR REGRESSION / LINEAR CLASSIFIER (WE HAVE A BORDER)

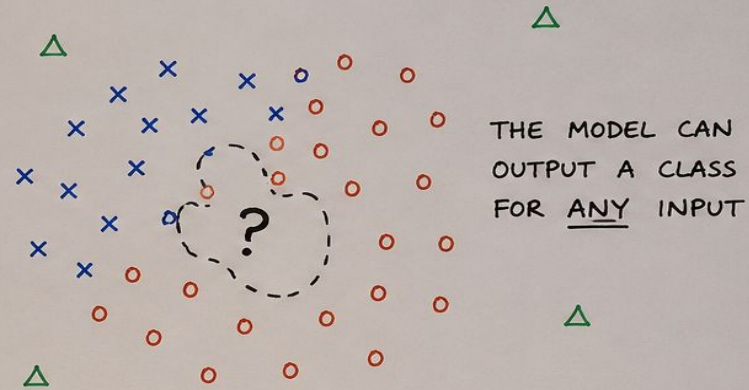


CLEAR SEPARATION

- BEST FIT LINE SEPARATES CLASSES
- WE CAN DEFINE WHICH SIDE BELONGS TO WHICH CLASS

x CLASS A o CLASS B Δ OOD EXAMPLE

NEURAL NETWORK (NO CLEAR BORDER) (NO GEOMETRIC BOUNDARY)



- NO EXPLICIT BORDER BETWEEN CLASSES
- THE MODEL ASSIGNS A CLASS TO EVERY POINT
→ EVEN TO POINTS FAR AWAY FROM THE TRAINING DATA (OOD)

KEY IDEA: LINEAR MODELS HAVE GEOMETRIC BOUNDARIES.
NEURAL NETWORKS DO NOT → THEY MAKE PREDICTIONS EVERYWHERE,
SO OOD DETECTION IS INHERENTLY DIFFICULT.

Problem Statement



Closed-World Assumption

Standard classifiers assume every test image belongs to a known class. A Cat-Dog model fed a bicycle must output Cat or Dog — it has no mechanism to abstain.



Softmax Overconfidence

Softmax always sums to 1.0. Even for a car image, the model outputs e.g. 'Dog — 87%'. This is a false confidence — the network extrapolates blindly beyond its training manifold.



Silent Failure

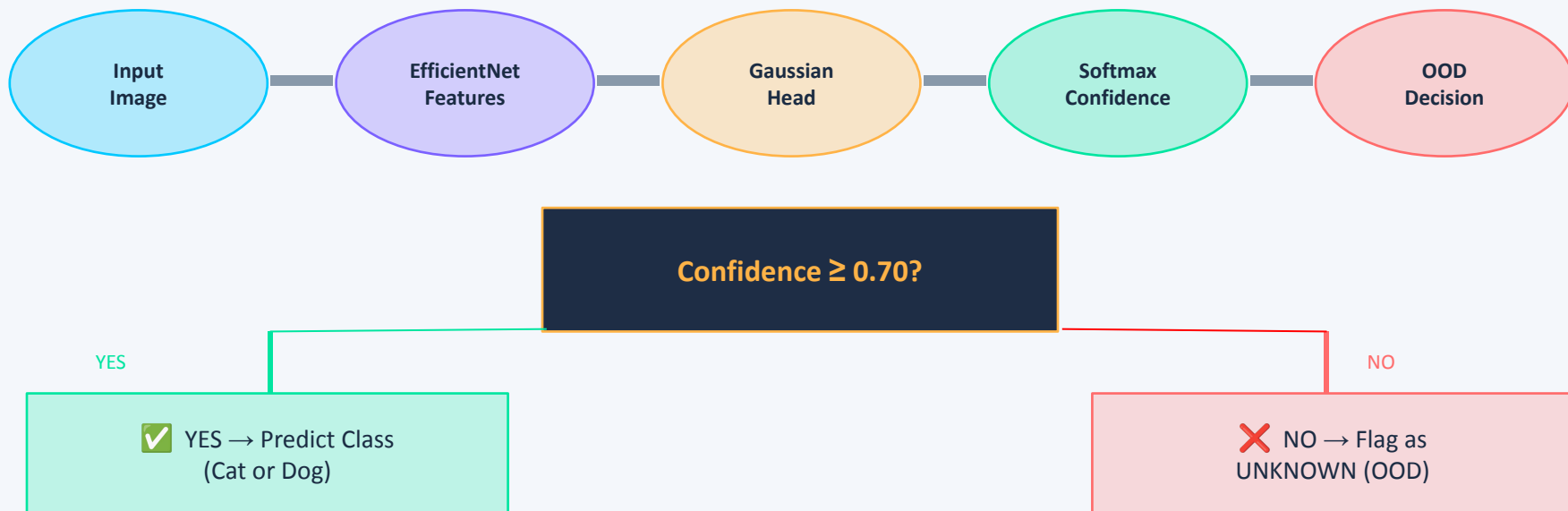
No error is raised. The prediction looks valid. In real systems (medical, safety-critical), this silent high-confidence misprediction is more dangerous than a visible failure.

Example Scenario:

A hospital uses a skin-lesion CNN. A patient uploads a photo of a bruise (not in training data). The model outputs 'Melanoma — 91% confident.' No alert is triggered. Diagnosis is wrong.

Project Objective and Proposed Solution

🎯 Objective: Build a Cat-vs-Dog classifier that correctly identifies and rejects images of unrelated objects (OOD inputs) rather than forcing a wrong prediction.



Baseline Model — EfficientNet-B0 + Linear Layer Head

Backbone

EfficientNet-B0 (pretrained, ImageNet)

Feature Dim

1280-D vector after AvgPool + Flatten

Frozen?

Yes — only the classifier head is trained

Optimizer

lr = 0.001, CrossEntropyLoss

Input Image $[3 \times 224 \times 224]$

EfficientNet-B0 Features [frozen]

AvgPool + Flatten $\rightarrow [1280]$

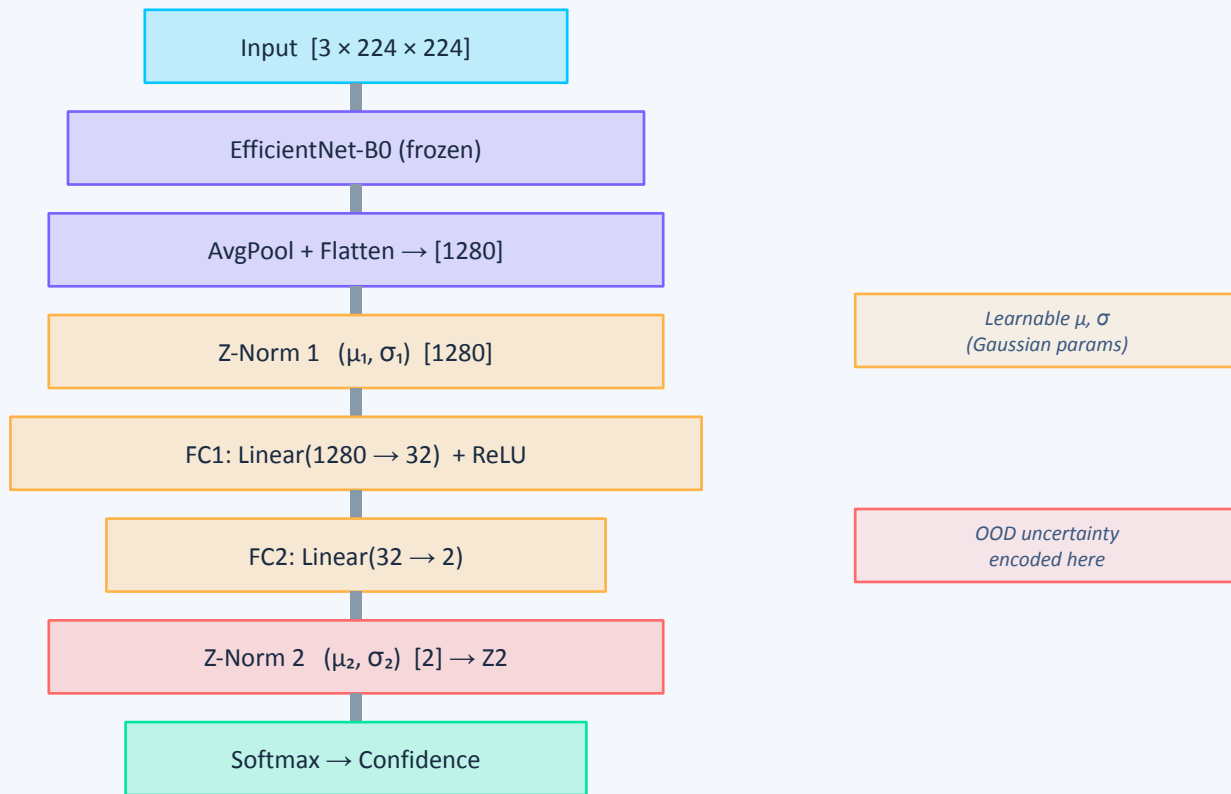
GaussianLayer: μ, σ $[2 \times 1280]$

Negative Distance Logits $[2]$

Softmax $\rightarrow$ Confidence

💡 Key Idea: GaussianLayer replaces the standard linear head with prototype-based classification. Each class has a learned center (μ) and spread (σ) in feature space. The negative Mahalanobis-like distance to the nearest prototype becomes the logit — making uncertainty naturally encoded in the output.

Custom CNN Architecture — EfficientNet-B0 + GaussianClassifier Head



OOD Detection Methodology — Confidence Threshold

How Confidence is Computed

1. Model outputs raw logits $Z_2 = [\text{logit_cat}, \text{logit_dog}]$
2. Softmax $\rightarrow$ probability distribution $[P(\text{Cat}), P(\text{Dog})]$ summing to 1.0
3. Confidence = $\max(P(\text{Cat}), P(\text{Dog}))$
4. If Confidence $< 0.70 \rightarrow$ flag as UNKNOWN (OOD)

Inference Flowchart



Key Code — GaussianClassifier Head (Custom CNN)

```
class GaussianClassifier(nn.Module):
    def
    __init__(self, in_features=1280,
              hidden_dim=
32, num_classes=2):
        self.mu1 = nn.Parameter(torch.zeros(in_features))

        self.sigma1 = nn.Parameter(torch.ones(in_features))

        self.fc1 = nn.Linear(in_features, hidden_dim)

        self.fc2 = nn.Linear(hidden_dim, num_classes)

        self.mu2 = nn.Parameter(torch.zeros(num_classes))

        self.sigma2 = nn.Parameter(torch.ones(num_classes))

    def
    forward(self, x):
        sigma1 = torch.abs(self.sigma1) + 1e-6

        z1 = (x - self.mu1) / sigma1
# Z-Norm input
        z = self.relu(self.fc1(z1))

        z = self.fc2(z)

        sigma2 = torch.abs(self.sigma2) + 1e-6

        z2 = (z - self.mu2) / sigma2
# Z-Norm output
        return z2
```



What it does

Defines a 2-layer MLP classifier wrapped in learnable Z-score normalization at input and output. Parameters μ and σ are trained alongside the network.



Why it matters

The model learns what 'normal' activations look like during training. At inference, OOD features produce abnormal Z-scores $\rightarrow$ uncertain (low) softmax confidence.



Role in pipeline

Replaces the EfficientNet classifier head. Its normalized output Z2 feeds Softmax to generate the confidence score used in OOD thresholding.

Key Code — OOD Detection Inference Loop

```
THRESHOLD = 0.7
```

```
for img_name in image_files:
```

```
    img = Image.open(img_path).convert('RGB')
```

```
    tensor = val_transform(img).unsqueeze(0).to(device)
```

```
    with torch.no_grad():
```

```
        feat = model.features(tensor)
```

```
        feat = torch.flatten(model.avgpool(feat), 1)
```

```
        z2 = model.classifier[1](feat)
```

```
        prob = F.softmax(z2, dim=1)
```

```
        conf, pred = torch.max(prob, 1)
```

```
    if
```

```
conf.item() < THRESHOLD:
```

```
    print(f'{img_name} -->
```

```
UNKNOWN (OOD) | Conf: {conf.item()*100:.1f}%')
```

```
    else:
```

```
        cls = class_names[pred.item()]
```

```
    print(f'{img_name} -->
```

```
{cls} | Conf: {conf.item()*100:.1f}%')
```



What it does

Runs inference on every image in the OOD folder. For each, it computes the softmax confidence score and compares it against the fixed threshold of 0.70.



Why it matters

This is the core OOD detection logic. Any image where $\max(\text{softmax}) < 0.70$ is flagged UNKNOWN. This prevents silent misclassification of non-cat/dog inputs.



Role in pipeline

Final inference stage. Outputs both: a classification label for in-distribution images, and OOD detection rate — the percentage of images correctly identified as unknown.

Results

Training Accuracy

95.73 %

Validation Accuracy

95.01 %

Detected 43 out of 98 images as OOD
OOD Detection Rate: 43.88%

OOD Detection Rate

43.88 %

<https://colab.research.google.com/drive/12nmPpbUub3paSZgXlct4lWRgJd6yVABp?usp=sharing>

https://colab.research.google.com/drive/1XY4eB-sflik-64o9H_5FjNDVLhq_T200?usp=sharing

Final model

Initial model (Based on Efficient Net)

<https://drive.google.com/drive/folders/1RnNxOIV4IKituOBtqskTtPVLDupeywDd?usp=sharing>
OOD Dataset

Advantages of Our Approach

Rejects the Unknown

Unlike a standard classifier that always outputs Cat or Dog, our model abstains when uncertain. Any non-cat/non-dog image is routed to 'UNKNOWN' rather than silently mislabelled.

Gaussian-Aware Uncertainty

The GaussianClassifier and GaussianLayer heads encode the learned distribution of in-distribution features. OOD inputs fall outside this distribution, naturally producing lower confidence scores.

Zero Extra Training Cost

OOD detection is entirely inference-time. No OOD training data is required, and no model retraining is needed. The threshold check is a single conditional statement.

Prototype Distance (GaussianLayer)

The baseline model uses explicit class-center distances (Mahalanobis-inspired). OOD inputs are far from both prototypes → both logits low → naturally uncertain softmax.

Easy to Deploy

The OOD check integrates into any PyTorch inference pipeline with minimal code changes. Works with any backbone — not specific to EfficientNet.

Interpretable

Confidence score is human-readable. A score of 0.52 clearly signals uncertainty; 0.95 signals high confidence. No additional post-processing or scoring layer needed.

Model Comparison

Metric	Baseline (EfficientNet + GaussianLayer)	Custom CNN (EfficientNet + GaussianClassifier)
Accuracy	99.14 %	95.73 %
OOD Detection Rate	2.04 %	43.88 %
OOD Method	Prototype Distance (Linear Layer)	Z-Norm MLP (Gaussian Classifier)
Backbone	EfficientNet-B0 (frozen)	EfficientNet-B0 (frozen)
Threshold	0.70	0.70

Discussion and Limitations

✓ STRENGTHS

1. No OOD training data required — zero extra annotation cost
2. Gaussian heads provide theoretically grounded uncertainty signal
3. Inference-time only — no retraining needed for deployment
4. Clean integration: one-line threshold check in any pipeline

⚠ LIMITATIONS

1. Threshold 0.70 is manually set — not calibrated on a labelled OOD validation set
2. Softmax overconfidence problem: some OOD inputs still get high confidence scores
3. Visually similar OOD (foxes, tigers) will fail silently — classified as Dog/Cat
4. No OOD-aware training loss — model not penalized for high OOD confidence

Future Scope

Energy-Based OOD

Replace softmax confidence with the free energy score of the logits. Theoretically more principled — energy is unbounded and more discriminative for OOD vs in-distribution.

Mahalanobis Distance

Fit a full class-conditional Gaussian per layer using the training set. Use the Mahalanobis distance with the full covariance matrix — more robust than our per-feature approximation.

Outlier Exposure

Explicitly train on labelled OOD data to push OOD confidence below the threshold. Directly teaches the model to be uncertain on unknowns — a major gap in the current approach.

Open-Set Recognition

Move beyond binary OOD flags — allow the model to describe what the unknown might be, using techniques like Class-Specific Residuals or Open-Set SoftMax.

More Classes & Larger Datasets

Extend beyond Cat-Dog to multi-class classification (breeds, species). Larger datasets like Oxford-IIIT Pets or iNaturalist would stress-test the OOD boundary more rigorously.

Real-Time Deployment

Quantize and export the model to ONNX or TorchScript. Deploy via FastAPI with a confidence-score endpoint. Integrate with mobile camera feeds for live OOD alerting.

Conclusion

This project demonstrates that standard binary classifiers can be extended with lightweight OOD detection using Gaussian-aware confidence thresholding — with no additional training data and no model retraining. Both the prototype-distance (GaussianLayer) and Z-normalized MLP (GaussianClassifier) heads provide a natural uncertainty signal that can be exploited at inference time to identify and reject unknown inputs.

🧠 Learned Gaussian parameters (μ , σ) encode in-distribution normality — OOD inputs fall outside this envelope

⚡ Confidence thresholding at 0.70 provides a simple, deployable OOD decision boundary requiring zero annotation

📐 Prototype-distance (GaussianLayer) is a theoretically motivated approach inspired by Mahalanobis distance

⚠️ Current limitations-uncalibrated threshold, no OOD training